LLMs from First Principles

Study Roadmap

Mathematics, models, machines and datacenters, from the ground up

Samadeep Sengupta
samadeep.github.io/study

October 2026

This is the plan the chapter PDFs follow. Each module lists what to learn, what to read, what to build, and the questions you must be able to answer before moving on.

Contents

The goal and the method **3**
What “done” looks like . . . 3
How to study each module . . . 3
The dependency map . . . 4
Suggested pace . . . 5

0 The math core **6**
0.1 Linear algebra . . . 6
0.2 Calculus and automatic differentiation . . . 6
0.3 Probability and statistics . . . 7
0.4 Information theory . . . 7
0.5 Optimisation . . . 8
0.6 Numerical computing . . . 8
0.7 Signal propagation . . . 9
0.8 Checkpoint . . . 9

1 Before the Transformer (chapter written) **10**

2 The Transformer **11**
2.1 Topics . . . 11
2.2 Read . . . 12
2.3 Lab . . . 12
2.4 Checkpoint . . . 12

2b Training and post-training **13**
3.1 Topics . . . 13
3.2 Read . . . 13
3.3 Lab . . . 13
3.4 Checkpoint . . . 14

3 GPUs and kernels **15**
3.1 Topics . . . 15
3.2 Read . . . 15
3.3 Lab . . . 16
3.4 Checkpoint . . . 16

4 Inference serving **17**
4.1 Topics . . . 17
4.2 Read . . . 17
4.3 Lab . . . 18
4.4 Checkpoint . . . 18

5 Parallelism and collectives **19**
5.1 Topics 19
5.2 Read 19
5.3 Lab 20
5.4 Checkpoint 20

6 Mixture of Experts **21**
6.1 Topics 21
6.2 Read 21
6.3 Lab 22
6.4 Checkpoint 22

7 Datacenter fabric and optics **23**
7.1 Topics 23
7.2 Read 23
7.3 Lab 24
7.4 Checkpoint 24

8 Capstone, the cost of a token **25**
8.1 Topics 25
8.2 Read 25
8.3 Output 25

Progress tracker **26**
Notes on sources 26

The goal and the method

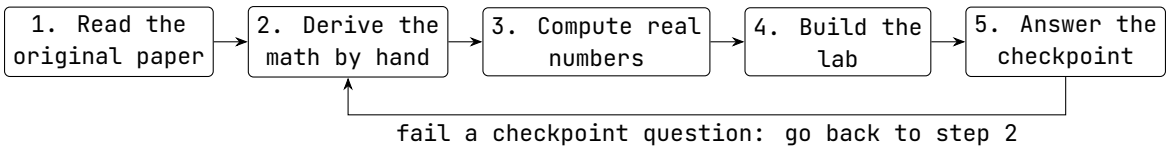
What “done” looks like

By the end you should be able to do each of the following on a whiteboard, with real numbers, without looking anything up:

1. **Derive** the training objective of a language model from the chain rule of probability, and explain cross-entropy, perplexity and bits per byte.
2. **Derive** backpropagation through a transformer block, including the Jacobians of softmax and RMSNorm, and explain why training costs about $6N$ FLOPs per token.
3. **Count** the parameters, FLOPs per token and KV-cache bytes per token of any published dense or MoE model from its config file.
4. **Explain** why decoding is memory-bandwidth bound and prefill is compute bound, using the roofline model and the specs of a real GPU.
5. **Design** a serving setup (batching, KV-cache management, quantisation, speculative decoding) for a latency target, and predict its throughput.
6. **Choose** a parallelism plan (data, tensor, pipeline, expert, context) for a given model and cluster, and estimate its communication volume per step.
7. **Explain** how an MoE layer routes tokens, how load balancing is enforced, and what the all-to-all costs on a real network.
8. **Sketch** a datacenter network for 10,000+ accelerators, and reason about where electrical packet switching and optical circuit switching each make sense.
9. **Compute** the cost of one generated token, end to end: compute, memory, network, power and money.

How to study each module

Every module follows the same loop. Do not skip steps 3 to 5; reading alone gives the feeling of understanding without the ability.



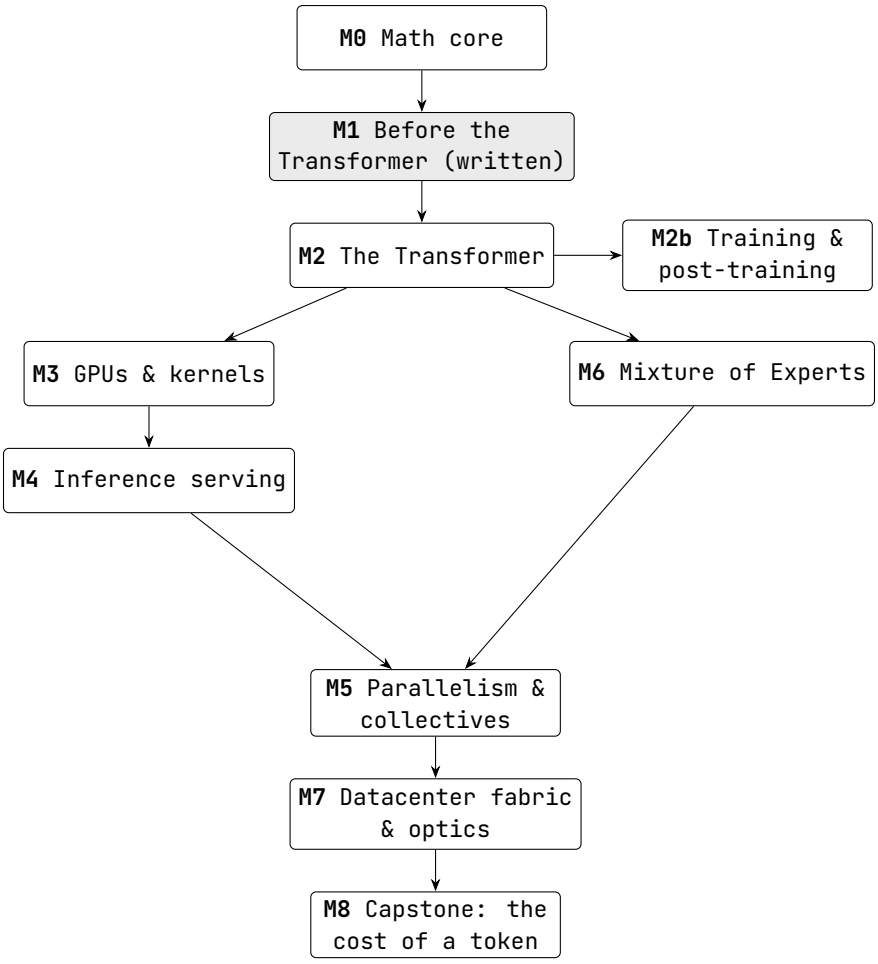
1. **Read the original paper**, not a blog summary. Summaries drop the assumptions, and the assumptions are where the understanding is.

2. **Derive** every equation the chapter uses, on paper. If you cannot derive it, you do not own it.
3. **Compute** with real model and hardware numbers. "Memory bound" means nothing until you have divided bytes by bandwidth yourself.
4. **Build** the small lab. Most labs run on a laptop (MLX on Apple silicon or plain NumPy is enough until Module 5).
5. **Checkpoint:** answer every question at the end of the module from memory. Any miss sends you back to step 2 for that topic.
6. **Blog cut** (optional): write one post per module about the single idea that surprised you most. Teaching is the final test.

Review. At the start of each new module, re-answer three checkpoint questions from older modules, chosen at random. This spaced review is what keeps Module 0 alive by the time you reach Module 7.

The dependency map

An arrow means "needs this first". The order below respects every arrow.



Suggested pace

Assuming about 6 to 8 focused hours a week. Adjust freely; the order matters more than the speed.

Module	Title	Weeks	Main output
M0	Math core	4	autodiff engine, float-format explorer
M1	Before the Transformer	2	chapter PDF ready; read + exercises
M2	The Transformer	3	a GPT from scratch, parameter counter
M2b	Training and post-training	2	SFT + DPO on a small model
M3	GPUs and kernels	3	roofline of your machine, tiled matmul
M4	Inference serving	3	KV cache + continuous batching server
M6	Mixture of Experts	2	MoE layer with load balancing
M5	Parallelism and collectives	3	collective cost simulator
M7	Datacenter fabric and optics	2	topology and optical-switch simulator
M8	Capstone	2	cost-per-token model, flagship post
	Total	≈ **26**	

Module 0

The math core

Goal. *Own exactly the mathematics later modules use, with each topic tied to the place it pays off.*

0.1 Linear algebra

Topics

- Vectors, dot products, norms, cosine similarity. *Pays off in:* attention scores, embeddings, retrieval.
- Matrix multiplication as composition of linear maps; shapes and einsum notation; why an $(m \times k)(k \times n)$ product costs $2mkn$ FLOPs.
- Rank and low-rank factorisation. *Pays off in:* LoRA, multi-head latent attention (MLA), the $q^\top k = h^\top W_Q^\top W_K h'$ view of attention.
- Eigenvalues, singular value decomposition, spectral norm, condition number. *Pays off in:* vanishing gradients, initialisation, quantisation error.
- Orthogonal and rotation matrices; complex numbers as 2-D rotations. *Pays off in:* deriving RoPE.
- Projections, subspaces, the residual stream as a shared vector space that every layer reads from and writes to.

Read

Strang, *Introduction to Linear Algebra* (chapters on SVD and orthogonality); Goodfellow, Bengio and Courville, *Deep Learning* (2016), ch. 2.

0.2 Calculus and automatic differentiation

Topics

- Multivariable chain rule, gradients, Jacobians, vector-Jacobian products.
- Computational graphs; forward-mode vs reverse-mode differentiation and their costs.
- Why one backward pass costs about two forward passes, giving the $\approx 6N$ FLOPs per training token rule ($2N$ forward, $4N$ backward).
- Jacobians by hand: softmax, cross-entropy, LayerNorm, RMSNorm, GELU/SiLU.

- Activation memory: what backprop must keep, and recomputation (gradient checkpointing) as a time-memory trade.

Read

Baydin et al., "Automatic Differentiation in Machine Learning: a Survey" (JMLR 2018); Rumelhart, Hinton and Williams (1986); Goodfellow et al., ch. 6.5.

Lab

A scalar autodiff engine in about 150 lines, then a tensor version; check every gradient against finite differences.

0.3 Probability and statistics

Topics

- Random variables, expectation, variance; variance of sums of independent terms. *Pays off in:* the $1/\sqrt{d}$ in attention, initialisation scales.
- Maximum likelihood estimation (it is the LM objective) and Bayes' rule.
- The categorical distribution and sampling: greedy, temperature, top-k, top-p (nucleus).
- Rejection sampling. *Pays off in:* the proof that speculative decoding is exact.
- Law of large numbers and concentration: why large-batch gradients are less noisy.

Read

Goodfellow et al., ch. 3; Holtzman et al., "The Curious Case of Neural Text Degeneration" (2020) for top-p.

0.4 Information theory

Topics

- Entropy, cross-entropy, KL divergence; $H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q)$.
- Perplexity, bits per token, bits per byte, and why comparisons need the same tokeniser.
- Prediction equals compression: arithmetic coding turns a model with cross-entropy H into a compressor using about H bits per symbol.
- Mutual information and PMI. *Pays off in:* word2vec, tokenisation.

Read

Shannon (1948); MacKay, *Information Theory, Inference, and Learning Algorithms* (2003), ch. 1 to 6; Cover and Thomas, *Elements of Information Theory*, ch. 2.

0.5 Optimisation

Topics

- Gradient descent, SGD, minibatch noise; momentum.
- Adam derived from first and second moment estimates with bias correction; AdamW and why decoupled weight decay differs from L2.
- Learning-rate warmup and cosine or WSD schedules; why warmup is needed with Adam.
- Batch size, the gradient noise scale and the critical batch size.
- Gradient clipping; loss spikes and their fixes (z-loss, QK-norm).
- Memory per parameter in mixed-precision Adam training: weights, gradients, master weights, two moments (about 16 bytes per parameter).

Read

Kingma and Ba, "Adam" (2015); Loshchilov and Hutter, "Decoupled Weight Decay Regularization" (2019); McCandlish et al., "An Empirical Model of Large-Batch Training" (2018); Wortsman et al., "Small-scale Proxies for Large-scale Transformer Training Instabilities" (2023).

0.6 Numerical computing

Topics

- IEEE 754 at the bit level: sign, exponent, mantissa; FP32, FP16, BF16, FP8 (E4M3, E5M2), FP4; range vs precision.
- Rounding error, overflow, underflow, catastrophic cancellation; loss scaling.
- The log-sum-exp trick and **online softmax** (computing a softmax in one streaming pass). *Pays off in:* FlashAttention, which is online softmax plus tiling.
- Quantisation: scale and zero point; per-tensor, per-channel, per-group; outlier features; stochastic rounding.

Read

Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic" (1991); Micikevicius et al., "Mixed Precision Training" (2018); Micikevicius et al., "FP8 Formats for Deep Learning" (2022); Milakov and Gimelshein, "Online Normalizer Calculation for Softmax" (2018).

Lab

A float-format explorer: encode and decode numbers in BF16, FP16 and FP8 by hand, and measure the error of summing 10^6 small numbers in each.

0.7 Signal propagation

Topics

- Xavier/Glorot and He initialisation, derived as variance preservation.
- Residual connections as identity paths; pre-norm vs post-norm; why deep transformers need them.
- LayerNorm vs RMSNorm.
- μP: hyperparameters that transfer across width (overview level).

Read

Glorot and Bengio (2010); He et al., "Delving Deep into Rectifiers" (2015); Ba et al., "Layer Normalization" (2016); Zhang and Sennrich, "Root Mean Square Layer Normalization" (2019); Xiong et al., "On Layer Normalization in the Transformer Architecture" (2020); Yang et al., "Tensor Programs V" (2022).

0.8 Checkpoint

1. Derive $\partial \ell / \partial z = p - e_y$ for softmax cross-entropy.
2. Why does the backward pass cost about twice the forward pass for a matmul?
3. If q, k have d independent unit-variance entries, what is $\mathrm{Var}(q^\top k)$? What follows for attention?
4. Write AdamW's update and explain each term.
5. What is the largest finite value in FP8 E4M3 and in BF16, and why does that matter for training vs inference?
6. Compute a softmax over a stream of numbers in one pass without overflow.

Module 1

Before the Transformer (chapter written)

Goal. *Understand the problems each architecture solved, from n-grams to attention.*

Covered

Language models and perplexity; n-grams and Kneser-Ney; Bengio's neural LM (2003); RNNs and vanishing gradients; LSTM and GRU; word2vec; sequence-to-sequence; Bahdanau and Luong attention; BPE; why the transformer won.

Checkpoint

The 13 exercises at the end of the Chapter 1 PDF.

Module 2

The Transformer

Goal. *Derive every component of a modern decoder-only transformer and count its cost exactly.*

2.1 Topics

- "Attention Is All You Need" line by line: scaled dot-product attention, multi-head attention, causal masking, the feed-forward block, residuals, normalisation.
- Encoder-only (BERT), encoder-decoder (T5) and decoder-only (GPT) families; why decoder-only dominates generation.
- Position: sinusoidal, learned, RoPE (derived as rotation), ALiBi; context extension by position interpolation, NTK scaling and YaRN.
- Modern block choices: SwiGLU, RMSNorm, pre-norm, no biases, tied embeddings.
- Attention variants that shrink the KV cache: multi-query, grouped-query, multi-head latent attention.
- Counting: parameters ($\approx 12Ld^2$ for a standard block stack), FLOPs per token, activation memory, KV-cache bytes per token, done for real configs.
- Scaling laws: Kaplan et al. (2020) and Chinchilla (2022); compute-optimal tokens per parameter; why later models over-train smaller models for cheaper inference.
- The mechanistic view: residual stream, induction heads (overview).
- Tokenisation in depth: byte-level BPE, SentencePiece, vocabulary size vs sequence length.

2.2 Read

Vaswani et al. (2017); Radford et al., GPT-1 (2018) and GPT-2 (2019); Brown et al., GPT-3 (2020); Devlin et al., BERT (2019); Raffel et al., T5 (2020); Su et al., RoFormer (2021); Press et al., ALiBi (2022); Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need" (2019); Ainslie et al., GQA (2023); DeepSeek-AI, DeepSeek-V2 (2024) for MLA; Shazeer, "GLU Variants Improve Transformer" (2020); Kaplan et al. (2020); Hoffmann et al. (2022); Touvron et al., Llama (2023); Llama 3 Herd of Models (2024); Chen et al., Position Interpolation (2023); Peng et al., YaRN (2023); Elhage et al., "A Mathematical Framework for Transformer Circuits" (2021).

2.3 Lab

A GPT from scratch (NumPy or MLX): train a character model on a small corpus, then a BPE model. A script that reads any Hugging Face config.json and prints parameters, FLOPs per token and KV bytes per token.

2.4 Checkpoint

1. Derive RoPE and show that $q_m^\top k_n$ depends only on $m - n$.
2. Count the parameters of Llama 3 8B from its config and match the published total.
3. KV-cache bytes per token for MHA, GQA with 8 groups, and MLA, for the same model width.
4. Under Chinchilla, how many tokens should a 70B model see? Why do labs train far beyond that?

Module 2b

Training and post-training

Goal. *Understand how a pretrained network becomes an assistant, and derive the objectives used.*

3.1 Topics

- Pretraining data: crawling, filtering, deduplication, mixtures.
- Pretraining stability: warmup, z-loss, QK-norm, loss-spike recovery.
- Supervised fine-tuning; parameter-efficient fine-tuning (LoRA).
- RLHF: reward modelling with the Bradley-Terry model; PPO with a KL penalty, derived.
- DPO, derived in closed form from the KL-regularised RLHF objective.
- RL with verifiable rewards; GRPO; reasoning models and test-time compute.
- Distillation.

3.2 Read

Christiano et al., “Deep RL from Human Preferences” (2017); Schulman et al., PPO (2017); Ouyang et al., InstructGPT (2022); Rafailov et al., DPO (2023); Shao et al., DeepSeekMath (2024) for GRPO; DeepSeek-AI, DeepSeek-R1 (2025); Hu et al., LoRA (2021); Hinton et al., “Distilling the Knowledge in a Neural Network” (2015); Lee et al., “Deduplicating Training Data Makes Language Models Better” (2022); Penedo et al., FineWeb (2024).

3.3 Lab

SFT then DPO on a small open model with a tiny preference set; plot the reward margin.

3.4 Checkpoint

1. Derive the DPO loss from the KL-constrained reward maximisation problem.
2. Why does PPO need a value function and GRPO not?
3. How many trainable parameters does LoRA rank 16 add to one 4096×4096 projection?

Module 3

GPUs and kernels

Goal. *Predict how fast a kernel can run on real hardware, and explain why.*

3.1 Topics

- Latency-oriented CPUs vs throughput-oriented GPUs; SIMT execution.
- Streaming multiprocessors, warps, threads, tensor cores; occupancy; Little's law for latency hiding.
- Memory hierarchy: registers, shared memory, L2, HBM; bandwidth and capacity at each level (taken from vendor whitepapers, not memory).
- **The roofline model**: arithmetic intensity, the ridge point, memory-bound vs compute-bound.
- Why decoding one token at batch 1 reads every weight once: tokens per second $\lesssim$ bandwidth / bytes of weights.
- CUDA programming model; tiled GEMM; kernel fusion; launch overhead; CUDA graphs.
- FlashAttention v1, v2, v3: tiling plus online softmax; IO complexity.
- Scale-up inside a node: NVLink, NVSwitch.

3.2 Read

Williams, Waterman and Patterson, "Roofline" (CACM 2009); Kirk and Hwu, *Programming Massively Parallel Processors*; NVIDIA CUDA C++ Programming Guide; NVIDIA Hopper and Blackwell architecture whitepapers; Volkov, "Better Performance at Lower Occupancy" (2010); Dao et al., FlashAttention (2022); Dao, FlashAttention-2 (2023); Shah et al., FlashAttention-3 (2024).

3.3 Lab

Measure the roofline of your own machine (peak FLOPs and bandwidth with MLX or Metal); then a naive vs tiled matmul and plot both on the roofline.

3.4 Checkpoint

1. Arithmetic intensity of a $(B \times d)(d \times d)$ matmul in BF16, as a function of B. At what B does it cross the ridge point of an H100?
2. Upper bound on tokens per second for a 70B BF16 model at batch 1 on one accelerator with 3.35 TB/s.
3. Why is FlashAttention faster although it does more FLOPs?

Module 4

Inference serving

Goal. *Design an LLM serving system for a latency and cost target and reason about its throughput.*

4.1 Topics

- Prefill vs decode; TTFT, TPOT (inter-token latency), throughput, goodput.
- KV-cache memory math; batch size as the decode lever; the throughput-latency curve.
- Batching: static, continuous (iteration-level), PagedAttention, prefix caching and radix trees, chunked prefill.
- Disaggregated prefill and decode.
- Speculative decoding: draft and verify, the proof of exactness, expected speedup; Medusa, EAGLE.
- Quantisation for serving: LLM.int8(), GPTQ, AWQ, SmoothQuant, FP8; KV-cache quantisation.
- Queueing theory basics: arrival rates, utilisation, Little's law, tail latency.
- Sampling and structured (grammar-constrained) decoding.
- Long-context serving: memory pressure, attention cost, offloading.

4.2 Read

Pope et al., "Efficiently Scaling Transformer Inference" (2022); Yu et al., Orca (OSDI 2022); Kwon et al., vLLM / PagedAttention (SOSP 2023); Zheng et al., SGLang (2024); Agrawal et al., Sarathi-Serve (OSDI 2024); Zhong et al., DistServe (OSDI 2024); Patel et al., Splitwise (ISCA 2024); Leviathan et al. (2023) and Chen et al. (2023), speculative decoding; Cai et al., Medusa (2024); Li et al., EAGLE (2024); Dettmers et al., LLM.int8() (2022); Frantar et al., GPTQ (2023); Lin et al., AWQ (2024); Xiao et al., SmoothQuant (2023); Willard and Louf, structured generation (2023).

4.3 Lab

A mini inference server: KV cache, continuous batching and a paged KV allocator in a few hundred lines; measure TTFT and TPOT as load grows. Then add a draft model and measure acceptance rate.

4.4 Checkpoint

1. KV-cache size for 64 concurrent requests at 8k context for a GQA 70B model in BF16. Does it fit beside the weights on 8 accelerators of 80 GB?
2. Prove that speculative decoding samples from the target distribution.
3. Why does chunked prefill reduce decode latency spikes?
4. Using Little’s law, how many requests are in flight at 20 req/s with 3 s mean latency?

Module 5

Parallelism and collectives

Goal. *Split a model across many accelerators and predict the communication each split costs.*

5.1 Topics

- Collectives from first principles: broadcast, reduce, all-reduce, all-gather, reduce-scatter, all-to-all; the α-β cost model.
- Ring vs tree all-reduce; the bandwidth-optimal $2\frac{p-1}{p}\frac{n}{B}$ result, derived.
- Data parallelism; ZeRO stages 1, 2, 3 and FSDP, with memory accounting for each.
- Tensor parallelism (column and row splits), sequence and context parallelism, Ring Attention.
- Pipeline parallelism: GPipe, 1F1B, interleaving, bubble fraction.
- Expert parallelism (bridge to Module 6).
- Choosing a 3-D or 4-D layout; overlapping communication with compute.
- Failures at scale: checkpointing frequency, stragglers.

5.2 Read

Thakur, Rabenseifner and Gropp, “Optimization of Collective Communication Operations in MPICH” (2005); Patarasuk and Yuan, “Bandwidth Optimal All-reduce Algorithms” (2009); Rajbhandari et al., ZeRO (2020); Zhao et al., PyTorch FSDP (2023); Shoeybi et al., Megatron-LM (2019); Narayanan et al., “Efficient Large-Scale Language Model Training on GPU Clusters” (2021); Huang et al., GPipe (2019); Narayanan et al., PipeDream (2019); Korthikanti et al., “Reducing Activation Recomputation” (2022); Liu et al., Ring Attention (2023); Jiang et al., MegaScale (2024); Austin et al., *How to Scale Your Model* (2025).

5.3 Lab

A collective-cost simulator: given p, message size, link bandwidth and latency, compare ring and tree all-reduce; then plug in a model and parallel layout to get time per step.

5.4 Checkpoint

1. Derive the ring all-reduce cost. Why is it independent of p for large p?
2. Memory per GPU to train a 70B model with Adam under ZeRO-3 on 64 GPUs (ignore activations).
3. Pipeline bubble fraction for 8 stages and 32 microbatches with 1F1B.
4. Which collective does tensor parallelism need per layer, and why is it kept inside a node?

Module 6

Mixture of Experts

Goal. *Understand sparse models from the 1991 origin to DeepSeek-V3, and what they cost to serve.*

6.1 Topics

- Origins: adaptive mixtures of local experts; hierarchical mixtures of experts and EM.
- The sparsely-gated MoE layer: noisy top-k gating, importance and load losses.
- GShard and Switch: top-2 and top-1 routing, the auxiliary load-balancing loss derived, capacity factor, token dropping.
- ST-MoE (router z-loss, stability); expert-choice routing; dropless MoE (MegaBlocks).
- Mixtral; DeepSeekMoE (fine-grained and shared experts); DeepSeek-V3 (auxiliary-loss-free balancing, node-limited routing).
- Economics: total vs active parameters; why MoE inference is memory and communication bound.
- All-to-all bytes per token for dispatch and combine, computed for a real model; expert-parallel serving; expert offloading.

6.2 Read

Jacobs, Jordan, Nowlan and Hinton, “Adaptive Mixtures of Local Experts” (1991); Jordan and Jacobs, “Hierarchical Mixtures of Experts and the EM Algorithm” (1994); Shazeer et al. (2017); Lepikhin et al., GShard (2021); Fedus et al., Switch Transformers (2022); Zoph et al., ST-MoE (2022); Zhou et al., Expert Choice routing (2022); Gale et al., MegaBlocks (2023); Rajbhandari et al., DeepSpeed-MoE (2022); Jiang et al., Mixtral (2024); Dai et al., DeepSeekMoE (2024); Wang et al., Auxiliary-Loss-Free Load Balancing (2024); DeepSeek-AI, DeepSeek-V3 Technical Report (2024).

6.3 Lab

An MoE layer with top-k routing and the Switch balancing loss; visualise expert load with and without the loss. Then compute all-to-all traffic per token for a published MoE config.

6.4 Checkpoint

1. Derive the Switch load-balancing loss and show its minimum is at uniform routing.
2. Active vs total parameters of Mixtral 8x7B and DeepSeek-V3; what each means for memory and for FLOPs.
3. Bytes sent per token in the dispatch all-to-all for top-8 routing with hidden size 7168 in FP8.

Module 7

Datacenter fabric and optics

Goal. *Reason about the network that connects tens of thousands of accelerators, including optical switching.*

7.1 Topics

- Clos networks (1953), fat-trees, leaf-spine; bisection bandwidth; oversubscription.
- Rail-optimised GPU cluster topologies; scale-up domains (NVLink/NVSwitch) vs scale-out fabric.
- RDMA; InfiniBand vs RoCE; lossless Ethernet with PFC; congestion control (DCQCN) and its failure modes.
- Google's Jupiter fabric and its move to optical circuit switches; the TPU v4 3-D torus and reconfigurable topology.
- Optical switching physics: MEMS mirrors, silicon photonics, other fast-switching approaches; insertion loss; reconfiguration time.
- Circuit vs packet switching; duty cycle as reconfiguration time vs burst length; rotor and nanosecond-switching research designs.
- Topology-aware training and topology reconfiguration for ML jobs.
- Power, cooling and PUE; failure rates at scale.

7.2 Read

Clos, "A Study of Non-Blocking Switching Networks" (1953); Leiserson, "Fat-Trees" (1985); Al-Fares et al., "A Scalable, Commodity Data Center Network Architecture" (2008); Singh et al., "Jupiter Rising" (2015); Poutievski et al., "Jupiter Evolving" (2022); Jouppi et al., TPU v4 (ISCA 2023); Zhu et al., DCQCN (2015); Guo et al., "RDMA over Commodity Ethernet at Scale" (2016); Gangidi et al., "RDMA over Ethernet for Distributed Training at Meta Scale" (2024); Mellette et al., RotorNet (2017); Ballani et al., Sirius (2020); Wang et al., TopoOpt (2023); Barroso, Hölzle and Ranganathan, *The Datacenter as a Computer*.

7.3 Lab

A fabric simulator: model a fat-tree and an optically reconfigured topology, replay an all-to-all pattern from Module 6, and plot duty cycle and completion time as reconfiguration time varies from milliseconds to nanoseconds.

7.4 Checkpoint

1. Number of switches and links in a 3-tier fat-tree built from k-port switches; hosts supported.
2. Why does PFC make Ethernet lossless, and what new failure does it introduce?
3. For a $20\,\mu$s burst, what reconfiguration time keeps a circuit switch above 90% duty cycle?
4. Which parallelism's traffic suits a slowly reconfigured topology, and which needs a fast one?

Module 8

Capstone, the cost of a token

Goal. *Tie everything together into one end-to-end model of a production LLM request.*

8.1 Topics

- Follow one request: arrival, scheduling, prefill, KV allocation, decode steps, MoE all-to-all, network hops, response.
- For a chosen model and cluster: compute, memory and bandwidth per token; achievable batch; latency.
- Power per token and dollars per million tokens; sensitivity to batch size, quantisation and context length.
- Where the next 10× comes from: hardware, algorithms, or the network.

8.2 Read

Pope et al. (2022); Austin et al., *How to Scale Your Model* (2025); the technical reports of whichever model you choose to cost.

8.3 Output

A spreadsheet or notebook cost model, and a flagship blog post built on it.

Progress tracker

Tick a box only when the checkpoint is answered from memory and the lab runs.

Module	Title	Started	Read	Derived	Lab	Checkpoint
M0	Math core		☐	☐	☐	☐
M1	Before the Transformer		☐	☐	n/a	☐
M2	The Transformer		☐	☐	☐	☐
M2b	Training and post-training		☐	☐	☐	☐
M3	GPUs and kernels		☐	☐	☐	☐
M4	Inference serving		☐	☐	☐	☐
M6	Mixture of Experts		☐	☐	☐	☐
M5	Parallelism and collectives		☐	☐	☐	☐
M7	Datacenter fabric and optics		☐	☐	☐	☐
M8	Capstone		☐	☐	☐	☐

Notes on sources

Every paper above is listed by author, short title and year so it can be found directly. Hardware numbers (bandwidths, FLOPs, link speeds) are deliberately left out of this roadmap except where used as exercise inputs; each module’s chapter will take them from the vendor whitepaper current at the time it is written.